Síntese dos artigos — Sistemas Operacionais

15 arquivos do pacote *OneDrive_1_11-11-2026.zip* · resumos em português para escolher qual ler por completo · 11/11/2026

Antes de escolher, observe:

- **08 e 09 são o mesmo trabalho** (arXiv 2407.14567): a 09 é a versão 1 (14 pág.) e a 08 é a versão 2, ampliada (47 pág.). Leia só um dos dois.
- **O arquivo 04 não é um artigo único:** são os anais completos do workshop OSPERT 2007 (82 pág., 9 artigos). O artigo que corresponde ao nome do arquivo (Gopalan & Kang) ocupa as págs. 48–57 dos anais.
- **O nome do arquivo 05 não bate com o conteúdo:** o nome fala de "containers na nuvem", mas o PDF é o survey *GPU Virtualization and Scheduling Methods* (2017). Containers aparecem só como direção futura.
- **O 07 é um capítulo de visão geral**, sem experimentos próprios.

Visão geral

#	Artigo	Tema	Tipo	Ano	Pág.	Dificuldade
01	QNodeOS – SO para nós de rede quântica	SO / computação quântica	Experimental (Nature)	2025	12	Alta
02	E/S assíncrona (io_uring)	E/S, syscalls	Discussão	2024	7	Média
03	CPU e memória para apps interativas de tempo real	Escalonamento + memória	Proposta + avaliação	2011	35	Média-alta
04	Alocação coordenada de múltiplos recursos (OSPERT)	Tempo real distribuído	Posição (anais)	2007	~10 de 82	Média
05	Virtualização e escalonamento de GPUs	Virtualização	Survey	2017	38	Média
06	H2M2 – memória heterogênea para LLMs	Memória / hardware / IA	Proposta + simulação	2025	22	Alta
07	Impacto da IA no projeto de SOs	IA + SO	Capítulo de visão geral	2025	10	Baixa
08	Integrando IA em SOs (v2)	IA + SO	Survey	2024	47	Média
09	SO e IA: revisão sistemática (v1)	IA + SO	Survey	2024	14	Média
10	STUN – ajuste do escalonador com RL	Escalonamento + IA	Proposta + avaliação	2022	16	Média
11	Saflo – escalonador MPTCP em eBPF	Redes / segurança	Proposta + avaliação	2025	15	Média-alta
12	Kernel Linux por verificação formal	Escalonador / métodos formais	Estudo de caso	2025	12	Alta
13	Virtuoso – simulação de memória virtual	Memória virtual	Ferramenta + validação	2025	22	Alta
14	Mensagens zero-copy em RTOS com CHERI	Sincronização / embarcados	Proposta + avaliação	2025	16	Média
15	vAttention – memória para servir LLMs	Memória virtual + IA	Proposta + avaliação	2025	18	Média-alta

Sínteses

01 QNodeOS: um sistema operacional para nós de rede quântica

Delle Donne, Iuliano, van der Vecht et al. (QuTech/TU Delft e outros) · *Nature* 639, mar. 2025 · 12 pág.

Experimental | Dificuldade: alta

O primeiro SO que permite programar aplicações de internet quântica em software de alto nível, independente do hardware físico.

Problema: até agora, cada experimento de rede quântica era programado "na mão", direto nos dispositivos de controle de baixo nível, por físicos, e fazia uma única tarefa.

Proposta: arquitetura que separa uma parte clássica (CNPU, que roda os programas e troca mensagens com outros nós), uma parte quântica (QNPU, com escalonador próprio, que executa blocos de instruções quânticas) e uma camada dependente do hardware (QDevice). Ela lida com escalas de tempo muito diferentes (ms na rede, µs nas operações quânticas, ns no controle) e com a vida curta dos qubits.

Resultados: rodou em dois nós com centros NV em diamante, inclusive uma computação delegada cliente-servidor, e ganhou um driver para um nó de íon aprisionado, mostrando que não depende da plataforma. Com multitarefa, a utilização do dispositivo quântico aumentou sem piorar a qualidade dos resultados.

Vale ler se: você quer ver conceitos clássicos de SO (processos, escalonador, abstração de hardware, multitarefa) aplicados num mundo novo. Tem trechos de física, e o código do SO não é aberto.

02 E/S assíncrona: com grandes poderes vêm grandes responsabilidades

Pestka, Paradies, Pohl (DLR / LMU Munique) · arXiv, nov. 2024 · 7 pág.

Discussão | Dificuldade: média | Mais curto

Por que APIs modernas de E/S assíncrona como o io_uring são tão rápidas e, mesmo assim, quase ninguém as usa.

Problema: SSDs modernos fazem milhões de operações por segundo com latência de microssegundos, e a E/S bloqueante tradicional (syscall e duas trocas de contexto por operação) desperdiça esse potencial.

Conteúdo: explica as filas em anel de submissão e de conclusão (io_uring, I/O Rings, SPDK), que trocam syscalls por operações atômicas em memória. O preço é que a aplicação passa a ter que escalonar suas próprias tarefas (multitarefa cooperativa). Compara arquiteturas para bancos de dados: *shared-nothing* (mais eficiente, mas pouco aplicável), pool fixo de threads de E/S (escala mal) e um pool dinâmico que liga e desliga instâncias de E/S conforme a carga.

Vale ler se: você quer algo curto e muito ligado ao conteúdo de aula (syscalls, troca de contexto, escalonamento, E/S). É o de melhor custo-benefício.

03 Escalonamento de CPU e gerência de memória para aplicações interativas de tempo real

Kato, Ishikawa, Rajkumar (Carnegie Mellon / Univ. de Tóquio) · *Real-Time Systems* (Springer), 2011 · 35 pág.

Proposta + implementação no Linux | Dificuldade: média-alta

Como manter vários vídeos tocando com taxa de quadros estável no Linux, mesmo com a CPU e a memória sob pressão.

Problema: com o escalonador de tempo real SCHED_DEADLINE (EDF + CBS), vídeos H.264 caíam abaixo de 10 fps quando mais de quatro rodavam juntos num quad-core, mesmo sem sobrecarga. E a paginação LRU mandava centenas de páginas do player para o disco quando outros processos consumiam memória.

Proposta: (1) FCBS, uma reserva de CPU mais flexível que redistribui tempo entre aplicações; (2) EDF-WM, escalonamento multicore que permite migrar tarefas entre núcleos; (3) PSAP, reserva de memória com um novo algoritmo de paginação que protege as aplicações de tempo real sem desperdiçar memória das demais.

Resultados: taxas de quadros maiores e mais estáveis que as do SCHED_DEADLINE, com custo extra desprezível, mais conjuntos de tarefas com prazo garantido e vídeos fluidos mesmo sob pressão de memória.

Vale ler se: você quer os dois temas centrais da disciplina (escalonamento e memória) juntos num problema concreto. É longo e tem bastante teoria de tempo real.

04 Alocação e escalonamento coordenados de múltiplos recursos em SOs de tempo real

Gopalan & Kang (SUNY Binghamton), dentro dos anais do OSPERT 2007 · ~10 pág. (o arquivo tem 82)

Artigo de posição | Dificuldade: média | Antigo

Sistemas distribuídos de tempo real precisam que o SO coordene CPU, rede e disco em conjunto, e não cada recurso isoladamente.

Problema: numa rede de câmeras de vigilância, por exemplo, cada quadro passa pela CPU (compressão), pela rede e pelo disco, e precisa cumprir um prazo de ponta a ponta. Os SOs de tempo real escalonam cada recurso sem saber dos outros.

Proposta (MURALS): dividir o prazo total entre as etapas equilibrando a carga; coordenar os escalonadores dos vários recursos durante a execução; dar garantias estatísticas em vez de reservar tudo para o pior caso; e usar controle por realimentação entre os recursos. É uma agenda de pesquisa, sem resultados completos.

Outros artigos no arquivo: drivers do Linux em escalonamento analisável, Feather-Trace (rastreamento leve), políticas aperiódicas no RT-Linux, OSEK/VDX, previsão de desempenho com variação de frequência, *lazy scheduling*, entre outros.

Vale ler se: você se interessa por tempo real distribuído. É de 2007 e é mais proposta do que resultado.

05 Virtualização e escalonamento de GPUs: um survey abrangente

Hong, Spence, Nikolopoulos (Queen's University Belfast) · *ACM Computing Surveys* 50(3), 2017 · 38 pág.

Survey | Dificuldade: média

Um mapa das técnicas para compartilhar GPUs entre várias máquinas virtuais na nuvem.

Conteúdo: explica por que virtualizar GPU é difícil (drivers fechados, arquiteturas sem padrão) e organiza as técnicas por nível: *API remoting* (interceptar chamadas CUDA/OpenCL na biblioteca), paravirtualização e virtualização completa (no driver) e virtualização assistida por hardware. Depois revisa escalonadores de GPU que buscam desempenho e justiça entre VMs.

Direções futuras: containers (virtualização leve, ainda pouco explorada para GPU na época), escalabilidade e outros aceleradores.

Vale ler se: você quer um panorama didático de virtualização de dispositivos. Por ser de 2017, não cobre tecnologias atuais (GPU em Kubernetes, particionamento moderno etc.).

06 H2M2: gerência de memória heterogênea por hardware para inferência de LLMs

Hwang, Kim, Lee, Kim, Huh (KAIST / Stanford) · arXiv, abr. 2025 · 22 pág.

Arquitetura de computadores | Dificuldade: alta

Combinar uma memória rápida e pequena (HBM) com uma grande e mais lenta (LPDDR) para rodar modelos de linguagem gigantes de forma mais barata.

Problema: LLMs precisam ao mesmo tempo de muita capacidade (pesos e cache KV, que cresce a cada token) e de muita largura de banda. Várias GPUs com HBM custam caro e gastam tempo se comunicando; usar a RAM do computador via PCIe é lento.

Proposta: memória assimétrica com aceleradores em cada tipo de memória, um algoritmo que decide em tempo real qual parte do modelo roda em qual memória (acompanhando o crescimento do cache KV) e uma abstração de memória em hardware para esconder essa complexidade.

Resultados: 1,46× (GPT-3 175B), 1,55× (Chinchilla-70B) e 2,94× (Llama2-70B) mais rápido que um sistema só com LPDDR; custo da abstração de no máximo 1,36%; 24% menos energia por token.

Vale ler se: você gosta de hardware e IA. É o mais "arquitetura de computadores" do pacote, com pouco de SO tradicional.

07 Impacto da IA no projeto de sistemas operacionais: uma visão geral

Pandikumar, Jakaraddi, Sevugapandi · Cap. 7 do livro *Shaping the Digital Future*, 2025 · 10 pág.

Visão geral | Dificuldade: baixa

Um passeio rápido pelos pontos do SO onde a IA pode entrar.

Conteúdo: lista usos de aprendizado de máquina no escalonamento (Q-Learning, DQN, PPO), na memória (LSTM para prever acesso a páginas, substituição de páginas com Q-learning, algoritmos genéticos contra fragmentação), na energia (DVFS guiado por previsão), na segurança (detecção de intrusão com Isolation Forest, SVM e LSTM), em interfaces adaptativas, na manutenção preditiva e em sistemas de arquivos.

Atenção: cita números como "até 30% menos faltas de página" e "50% menos fragmentação" sem experimentos próprios nem fonte específica. É raso comparado aos surveys 08/09.

Vale ler se: você quer uma introdução de uma hora ao vocabulário. Não é boa escolha para um trabalho aprofundado.

08 Integrando IA em sistemas operacionais: survey abrangente (versão 2)

Zhang, Zhao, Li, Yin, Zhang, Chen (Univ. de Zhejiang e outros) · arXiv 2407.14567v2, dez. 2024 · 47 pág.

Survey | Dificuldade: média | Mesmo trabalho do 09

A revisão mais completa do pacote sobre como IA e SO se combinam.

Conteúdo: parte de 824 artigos (ACM, IEEE, arXiv e Google Scholar). Revisa o aprendizado de máquina aplicado a escalonamento, armazenamento, memória, rede e segurança; o uso de LLMs no desenvolvimento do SO (por exemplo, gerar extensões do kernel a partir de linguagem natural) e na interação com o usuário; e arquiteturas que favorecem um "SO inteligente" (kernel bypass, Library OS). Propõe estágios de evolução até um SO guiado por IA.

Conclusão: a IA já melhora funções do SO, mas faltam conjuntos de dados maiores, treinamento eficiente, explicabilidade e validação robusta.

Vale ler se: você vai escrever sobre "IA + SO" e precisa de base. É longo, mas dá para ler só as seções de interesse.

09 Sistema operacional e inteligência artificial: revisão sistemática (versão 1)

Mesmos autores do 08 · arXiv 2407.14567v1, jul. 2024 · 14 pág.

Survey | Dificuldade: média | Mesmo trabalho do 08

A primeira versão, mais enxuta, do mesmo trabalho do item 08.

Conteúdo: 108 estudos (2019 a mar. 2024), organizados em cinco perguntas: quais partes do SO a IA mais melhora (AI4OS); como o SO pode servir melhor a sistemas de IA (OS4AI); quais técnicas de IA são usadas; quais arquiteturas novas ajudam (com destaque para Library OS); e como os LLMs abrem oportunidades para o SO.

Diferença para o 08: é mais curto e trata explicitamente da via de mão dupla (OS4AI). O 08 amplia a busca e aprofunda LLMs e os estágios do SO inteligente.

Vale ler se: você quer o tema do 08 em um terço do tamanho.

10 STUN: otimização dos parâmetros do escalonador do kernel por aprendizado por reforço

Lee, Jung, Jo (Chungbuk National University, Coreia) · *Applied Sciences* (MDPI), 2022 · 16 pág.

Proposta + avaliação | Dificuldade: média (acessível)

Um agente de aprendizado por reforço ajusta sozinho o escalonador do Linux para uma carga de trabalho específica.

Problema: quase todas as máquinas usam os parâmetros padrão do escalonador do Linux (CFS), pensados para uso genérico. Ajustá-los à mão exige muito conhecimento, e os parâmetros interagem entre si.

Proposta: o STUN ajusta automaticamente as 5 políticas de escalonamento e 10 parâmetros do kernel com Q-learning. Um filtro descarta parâmetros que não afetam a carga (acelerando o treino) e a recompensa é refinada. O foco são cargas repetitivas, como jobs em lote.

Resultados: +27,7% no Hackbench; detecção de faces com 18,3% menos tempo e 22,4% mais FPS; Sysbench com +27%, +54% e +256% em máquinas de 4, 44 e 120 núcleos.

Vale ler se: você quer ver IA e escalonamento juntos de forma concreta e fácil de entender. Ótimo para apresentar em sala.

11 Saflo: escalonador MPTCP em eBPF contra ataques de análise de tráfego em redes celulares

Lee, Jin, Stoleru (Texas A&M) · arXiv, fev. 2025 · 15 pág.

Proposta + testbed LTE/5G | Dificuldade: média-alta

Espalhar o tráfego por vários caminhos (LTE, 5G, Wi-Fi) de forma imprevisível para que um espião não descubra qual vídeo você assiste.

Problema: as mensagens de controle DCI do LTE/5G (que dizem quando e quanto dado vai para cada usuário) não são criptografadas. Ataques já usam isso para identificar vídeos e detectar a presença de usuários, e o escalonador MPTCP padrão do Linux (BLEST) não protege.

Proposta: um escalonador MPTCP no kernel escrito em eBPF, mais programas em espaço de usuário que decidem aleatoriamente quais caminhos ficam ativos e rodam um detector de ataques com redes neurais. Os dois lados se comunicam por mapas eBPF.

Resultados: o acerto do ataque de identificação de vídeo cai de 96,9% para 12,4%, e o de identificação de usuário de 99,4% para 59,5%. O desempenho de rede fica melhor que o de um caminho único.

Vale ler se: você quer um bom exemplo de eBPF (estender o kernel com segurança) e de cooperação entre kernel e espaço de usuário. Exige alguma base em redes.

12 Entendendo código do kernel Linux por verificação formal

Lawall, Nishimura, Lozi (Inria Paris) · OLIVIERFEST '25 (workshop ACM), 2025 · 12 pág.

Estudo de caso | Dificuldade: alta

Usar provas formais como ferramenta para realmente entender uma função do escalonador do Linux.

Problema: o escalonador do Linux tem mais de 34 mil linhas e pouquíssimos desenvolvedores o entendem por completo.

Conteúdo: os autores escrevem pré e pós-condições e invariantes de laço e provam com Frama-C a função *select_idle_core* (versão de 2016), que procura um núcleo ocioso para acordar uma tarefa. Também estudam como as versões dos iteradores de máscara de CPU afetam a prova.

Achados: o processo revelou um comportamento possivelmente indesejado que ainda existe no Linux 6.15 e uma refatoração provavelmente errada que sobreviveu de 2009 a 2022. As provas são trabalhosas e ficam desatualizadas rápido, porque o kernel muda muito.

Vale ler se: você gosta de métodos formais e de código real do kernel. Exige familiaridade com C e lógica.

13 Virtuoso: pesquisa rápida e precisa em memória virtual

Kanellopoulos et al. (ETH Zürich, grupo de Onur Mutlu) · ASPLOS 2025 · 22 pág.

Ferramenta de simulação | Dificuldade: alta

Um "mini SO" que imita só a parte de memória virtual do kernel, para testar ideias novas de hardware e SO rapidamente em simulação.

Problema: a tradução de endereços pode consumir até 40% do tempo de programas longos, e a alocação de memória até 95% do tempo de programas curtos. Mas os simuladores ou ignoram o SO (rápidos e imprecisos) ou simulam o sistema inteiro (precisos, lentos e difíceis de modificar).

Proposta: o MimicOS, um kernel leve em espaço de usuário, escrito em linguagem de alto nível, que imita apenas as rotinas de memória virtual (faltas de página, alocação, páginas grandes, tabelas de páginas). Foi integrado a cinco simuladores.

Resultados: modela a MMU de um processador de servidor real com 82% de precisão e a latência de falta de página do Linux com até 79%, com só 20% de custo extra de simulação. O código é aberto.

Vale ler se: você quer aprofundar memória virtual (TLB, tabela de páginas, páginas grandes). É denso e voltado para pesquisa.

14 Mensagens zero-copy: comunicação entre tarefas de baixa latência em RTOS com CHERI

Soltani Siapoush, Alves-Foss (University of Idaho) · *Future Internet* (MDPI), nov. 2025 · 16 pág.

Proposta + avaliação | Dificuldade: média

Trocar mensagens entre tarefas de um sistema embarcado sem copiar dados e sem abrir mão do isolamento de memória.

Problema: em RTOS como o FreeRTOS, a comunicação entre tarefas (filas, mutexes, semáforos) passa pelo kernel e copia os dados várias vezes, e não há isolamento de memória entre tarefas.

Proposta: usar o hardware CHERI, cujos ponteiros ("capabilities") carregam limites e permissões verificados pelo próprio hardware. Mutexes e semáforos viram capabilities seladas, as mensagens passam por um buffer circular compartilhado sem cópia, e capabilities com prazo de validade evitam o uso de memória já liberada.

Resultados: no CHERIoT RTOS, travar um mutex ficou até 3× mais rápido e a troca de mensagens mais de 70% mais rápida que no FreeRTOS, mantendo o comportamento de tempo real e resistindo a acesso indevido e a ataques TOCTTOU.

Vale ler se: você quer juntar sincronização (mutex, semáforo, produtor-consumidor), proteção de memória e tempo real. É recente e relativamente acessível.

15 vAttention: gerência dinâmica de memória para servir LLMs sem PagedAttention

Prabhu, Nayak, Mohan, Ramjee, Panwar (Microsoft Research Índia) · ASPLOS 2025 · 18 pág.

Proposta + avaliação | Dificuldade: média-alta

A ideia clássica de memória virtual do SO (separar o endereço virtual da memória física) aplicada para servir modelos de linguagem na GPU.

Problema: o cache KV de cada requisição cresce token a token e seu tamanho final é desconhecido. O PagedAttention (vLLM), padrão de mercado, evita fragmentação alocando blocos sob demanda, mas deixa o cache espalhado na memória virtual. Isso obriga a reescrever os kernels de atenção e os deixa mais lentos (até 2,8× em alguns casos).

Proposta: reservar memória virtual contígua e só associar memória física quando for preciso, com as APIs de memória virtual do CUDA (paginação sob demanda, como num SO). Para esconder a latência, sobrepõe alocação e computação e aloca páginas antecipadamente. Também altera o driver para usar páginas de 64 KB em vez de 2 MB, reduzindo a fragmentação.

Resultados: funciona com kernels de atenção existentes sem modificação e aumenta a vazão em até 1,23× em relação às versões com PagedAttention do FlashAttention-2 e do FlashInfer.

Vale ler se: você quer ver conceito de livro-texto (paginação, fragmentação, memória virtual) resolvendo um problema atual de IA. Muito recomendado.

Guia rápido de escolha

- **Quero algo curto:** 02 (7 pág.) ou 09 (14 pág.).
- **Quero IA + SO com resultados concretos:** 15 (vAttention) ou 10 (STUN).
- **Quero um panorama para fundamentar um trabalho:** 08, ou 09 se quiser mais curto.
- **Gosto de escalonamento e tempo real:** 03 (clássico e completo), 14 (moderno, embarcados) ou 04 (proposta antiga).
- **Gosto de memória:** 15 (aplicado), 13 (pesquisa com simulação) ou 06 (hardware).
- **Gosto de segurança e redes:** 11 (eBPF + 5G) ou 14 (CHERI).
- **Quero algo diferente de tudo:** 01 (SO para rede quântica, publicado na Nature).
- **Gosto de métodos formais:** 12.
- **Evite como leitura principal:** 07 (superficial) e 04 (antigo, e o arquivo é um conjunto de artigos). O 05 só se não precisar de algo atual.

> **Se for para escolher um só:** o **15 (vAttention)** tem o melhor equilíbrio. É recente, foi publicado num evento de ponta (ASPLOS 2025), tem tamanho razoável e liga diretamente a matéria de memória virtual a um problema atual. Alternativas: o **10 (STUN)**, se preferir algo mais simples, e o **02**, se o tempo estiver curto.